Programación Avanzada
IIC2233 2025-1

Pablo Araneda - Daniela Concha - Francisca Ibarra - Lucas Van Sint Jan - Francisca Cattan

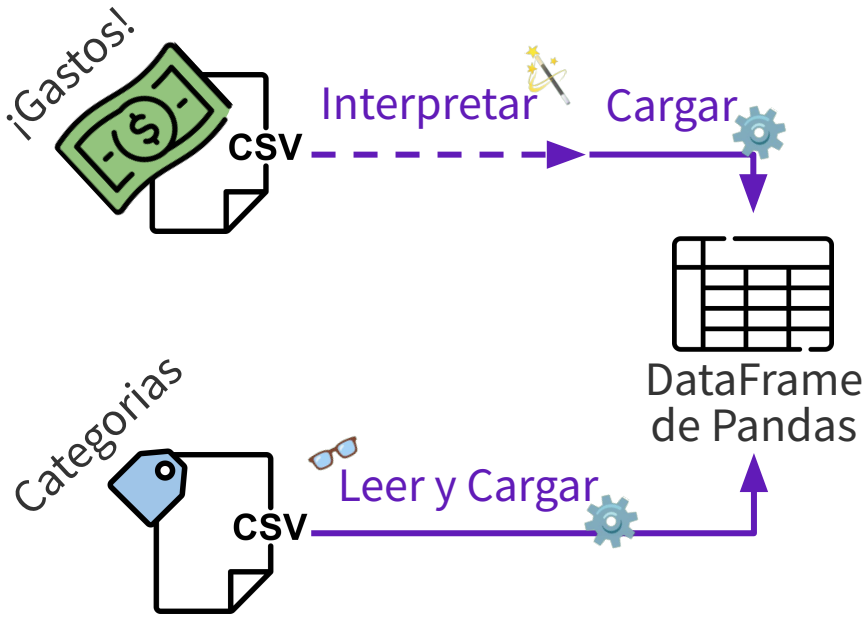
💸 Experiencia 3 💸 DCCuida tus gastos

Fascinado por las actividades de transacciones y división de cuentas, procedes a crear un sistema que **almacena distintos gatos y las categorías** de estos mismos.

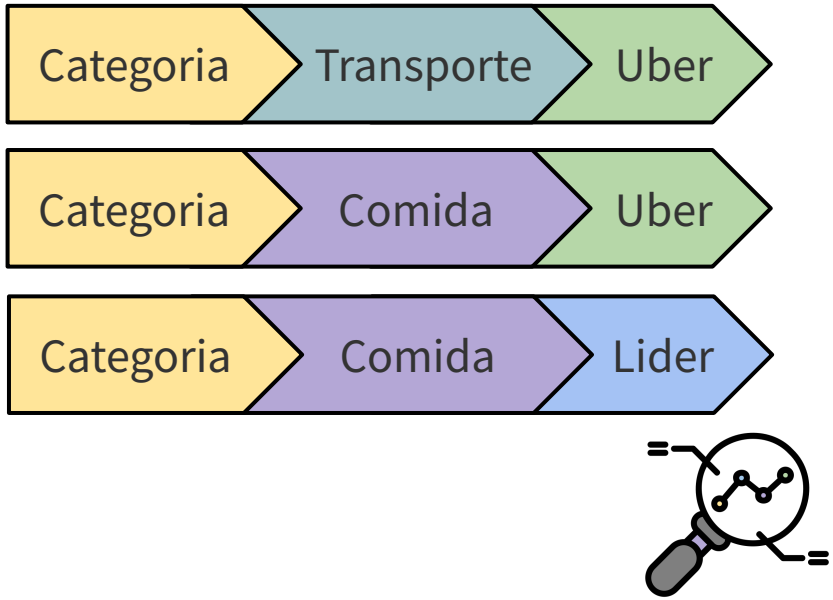
Los datos fueron corrompidos y no sabes cómo manejar las categorías, trabajemos juntos para resolver estos problemas 💪.

¿Qué queremos lograr?

Cargar base de datos: Gastos y Categorías



Revisar relaciones entre Categorías



💸 Resumen ejecutivo 💸

1. Identificaremos la información un archivo corrupto utilizando sólo **Expresiones regulares**.
2. Cargaremos la información del archivo corrupto y un archivo CSV en **DataFrames de Pandas**. Además, operaremos sobre el DataFrame para revisaremos la información cargada.
3. Transformaremos la información de las categorías en un **Grafo** y lo **recorreremos utilizando distintos algoritmos**.

Al igual que con las experiencias anteriores, trabajaremos **en partes.** Al final del día se publicará la solución donde se detalla cada método del código correctamente programado, y en caso de tener errores, cuáles eran estos.

¿Qué tenemos?

¿Qué tenemos?

✅ Método
❗ Incompleto o no implementado

gastos.csv

categorias.csv

main.py

```
❗ interpretar_archivo_regex(path) -> list[dict]

❗ cargar_gastos(path) -> df
❗ cargar_categorias(path) -> df
✅ combinar_gastos_categorias(df1, df2) -> df
❗ describir_informacion(df) -> None

❗ crear_grafo_categorias(df) -> ???
❗ recorrer_dfs(grafo) -> None
❗ recorrer_bfs(grafo) -> None
```

¡A trabajar!

Parte I: Interpretar archivo corrupto 🪄

Actualmente…

- El archivo “gastos.csv” está corrupto y los datos de una fila no presentan un orden fijo. Además, la función “**interpretar_archivo_regex()”** está incompleta.

Resultado esperado tras terminar

- Hacer una lista de diccionarios con la información del archivo “gastos.csv”, donde el diccionario contenga:
  - **id_gasto** Identificador de un gasto.
  - **monto** Dinero asociado al gasto.
  - **fecha** Fecha asociada al gasto.

Parte I: Interpretar archivo corrupto 🪄

1. Observaremos los datos 👀. Trataremos de **identificar el formato** de cada uno de los campos de un gasto: id_gasto, monto, fecha.
2. Crearemos **tres regex**, donde cada una estará asociada a un campo. Estas regex deberán usar **grupos de captura con nombre** para así poder extraer la información que encontremos.
3. Leeremos el contenido del archivo “gastos.csv” y usando el módulo **re** crearemos una lista de diccionarios con la información de cada gasto.

¡A programar! 💻 🔧

Torpedo: Expresiones Regulares

Sintaxis	Descripción
`[ ]`	Clases de caracteres
`[a-z]`	Cualquier letra minúscula entre a y z
`\d`	Cualquier dígito entre 0 y 9
`\`	Escapar meta-carácter

Sintaxis	Descripción
`+`	Está 1 o más veces
`*`	Está 0 o más veces
`?`	Está o no está
`{m, n}`	Está entre m y n veces, inclusive
`{m}`	Está m veces

Sintaxis	Descripción
`|`	Operador **OR**
`( )`	Grupo o Grupo de captura
`(?P<id> )`	Grupo de captura **con nombre**

Sintaxis	Descripción
`^`	Inicio del *string*
`$`	Fin del *string*

Parte II.a: Cargar información gastos

Actualmente…

- Tenemos la información del archivo “gastos.csv”, pero la función “**cargar_gastos()**” está incompleta.

Resultado esperado tras terminar

- Crear un DataFrame a partir de la información del archivo “gastos.csv”.

	id_gasto	monto	fecha
0	a2345678	$1000	2025-06-06 16:16
1	a2345679	$5590	2024-04-29 20:19
		□	

¡A programar! 💻🔧

Parte II.b: Cargar información categorías

Actualmente…

- Tenemos las categorías en un archivo CSV, pero la función “**cargar_categorias()**” está incompleta.

Resultado esperado tras terminar

- Crear un DataFrame a partir de la información del archivo “categorias.csv”.

	id_gasto	categorias
0	a2345678	Categoria>Transporte>Uber
1	a2345679	Categoria>Comida>Lider
	□	

¡A programar! 💻🔧

Parte II.c: Combinar información

Actualmente...

- Tenemos la **información dividida en dos DataFrames** (gasto y categorías) y la función "**combinar_gastos_categorias()**" se encuentra completa.

	id_gasto	monto	fecha	categorias
0	a2345678	$1000	2025-06-06 16:16	Categoria>Transporte>Uber
1	a2345679	$5590	2024-04-29 20:19	Categoria>Comida>Lider
			□	

¡Veamoslo en acción! 💻

¡A programar! 💻🔧

Parte II.d: Describir información

Actualmente…

- Tenemos toda la **información en un DataFrame**, pero la función "**describir_informacion()**" se encuentra incompleta.
- Si revisamos el tipo de dato, **todas las columnas son "objects" (*strings*)**, ¿está bien eso?

Resultado esperado tras terminar

- Actualizar los tipos de datos
  - Pasemos "monto" a int.
  - Pasemos "fecha" a datetime.
    *Ayuda
- Revisar los datos cargados:
  - Tipo de dato de cada columna.
  - Porte de la matriz.
  - Distribución de la información cuantitativa.
  - etc.

Parte III.a: Abstraer Categorías

Actualmente…

- Las categorías de cada gasto presentan un orden jerárquico. Esta información se encuentra columna del DataFrame, pero se está contenida en un único *string*. Además, la función “**crear_grafo_categorias()**” se encuentra incompleta.

Resultado esperado tras terminar

- Representar la relación entre las jerarquías en un grafo.

Parte III.a: Abstraer Categorías

Antes de ir al código:

1. Comprendamos **cómo funciona la jerarquía de las categorías**, para esto revisemos el archivo el CSV o nuestro DataFrame.
2. ¿Cómo podemos **representar las relaciones de categorías**? ¿Alguna es más conveniente que otra?

¡A programar! 💻 🔧

Parte III.b: Recorrer grafo de Categorías

Actualmente…

- Tenemos toda la **información en un Grafo**, pero las funciones “**recorrer_dfs()**” y “**recorrer_bfs()**” se encuentran incompletas.

Resultado esperado tras terminar

- Poder recorrer el grafo con ambos algoritmos.

¡A programar! 💻 🔧

¡Lo logramos!

Resumen de lo que hicimos

1. Identificamos la información un archivo corrupto utilizando sólo **Expresiones regulares**.
2. Cargamos la información en **DataFrames de Pandas** y operamos sobre estos.
3. Transformaremos las categorías con en un **Grafo** y lo **recorrimos utilizando BFS y DFS**.

¿Qué más podemos hacer?

1. Guardar la información corregida.
2. Explorar la información con mayor detalle.
3. Utilizar otra estructura para almacenar la información del grafo.
4. Implementar las funciones para recorrer los grafos de forma recursiva.
5. Y mucho más… 🤓

Programación Avanzada
IIC2233 2025-1

Pablo Araneda - Daniela Concha - Francisca Ibarra - Lucas Van Sint Jan - Francisca Cattan